

Uso de Inteligencia Artificial: Para el presente taller, estará permitido el uso de Inteligencia Artificial (IA), siempre y cuando se enlace la conversación o se adjunte la discusión que tuvo con la IA seleccionada como un anexo al documento entregable.

Conversaciones con Gemini utilizadas para las gráficas del punto 1:

Hola. Ayúdame a mejorar mi gráfico. Quiero que sea interactivo (al hacer hover me muestre el valor de la variable IDF_dep y de la variable Rango_Cat), que incluya título (Índice de Desempeño Fiscal por Departamento - 2024), leyenda (la escala usada para el IDF_dep), escala de colores (Entre más alto verde y entre más bajo rojo), y una nota que diga fuente (DNP). Adicionalmente, haz que no haya leyendas en los ejes, que no haya grid (fondo blanco) y que las líneas de división geográficas sean muy delgadas en gris claro

```
IDF_DPTOS <- ggplot() +  
  geom_sf(data = sf_data, aes(fill = IDF_dep))
```

Para lograr un mapa interactivo con las especificaciones que buscas, la mejor opción en R es combinar ggplot2 con plotly. La función ggplotly() convertirá tu objeto estático en uno interactivo de forma casi automática.

Aquí tienes el código ajustado:

```
R  
library(ggplot2)  
library(plotly)  
library(scales) # Para manejar las escalas de colores  
  
# 1. Creamos el objeto ggplot  
p <- ggplot(data = sf_data) +  
  geom_sf(aes(fill = IDF_dep,  
    # El argumento 'text' define lo que aparecerá en el hover  
    text = paste("Departamento:", DPTO_CNMBRE, # Asumiendo nombre del depto  
      "<br>IDF:", IDF_dep,  
      "<br>Rango:", Rango_Cat)),  
    color = "lightgray",  
    size = 0.1) + # Líneas geográficas delgadas y gris claro  
  
# Escala de colores: Rojo (bajo) a Verde (alto)  
scale_fill_gradient(low = "red", high = "green", name = "IDF") +  
  
# Título y Notas  
labs(title = "Índice de Desempeño Fiscal por Departamento - 2024",  
  caption = "Fuente: DNP") +  
  
# Configuración estética del fondo y ejes  
theme_void() + # Elimina ejes, fondos, grids y leyendas de ejes de golpe  
theme(  
  panel.background = element_rect(fill = "white", color = NA),  
  plot.title = element_text(hjust = 0.5, face = "bold"),  
  legend.position = "right"  
)  
  
# 2. Convertimos a interactivo con plotly
```

```
# 'tooltip' indica que use la estética 'text' definida arriba
mapa_interactivo <- ggplotly(p, tooltip = "text") %>%
  layout(title = list(text = paste0('Índice de Desempeño Fiscal por Departamento - 2024',
    '<br>',
    '<sup>Fuente: DNP</sup>'))))
```

```
# Visualizar
mapa_interactivo
```

Detalles clave de la configuración:

- **Interactividad (text):** Dentro del aes(), definí un parámetro text. Esto permite que al pasar el mouse (hover), se despliegan exactamente las variables que pediste (IDF_dep y Rango_Cat) con un formato limpio.
- **Limpieza Visual:** Usé theme_void(). Es el tema más eficiente para mapas porque elimina automáticamente los números de latitud/longitud, las etiquetas de los ejes y el grid, dejando el fondo totalmente limpio.
- **Líneas de división:** En geom_sf, el parámetro color = "lightgray" y size = 0.1 (o linewidth = 0.1 dependiendo de tu versión de ggplot2) asegura que los límites departamentales no distraigan del color principal.
- **Escala de colores:** scale_fill_gradient crea la transición directa entre rojo y verde. Si prefieres un punto medio (amarillo), podrías usar scale_fill_gradient2 o scale_fill_viridis_c(option = "RdYlGn").

Nota: Asegúrate de que las variables IDF_dep y Rango_Cat estén correctamente escritas en tu objeto sf_data para que plotly las reconozca al generar la etiqueta flotante.

Ahora ayúdame a crear otro mapa, pero con fill de la variable Rango_Cat en vez de IDF_dep, la escala tendrá solo 5 valores posibles que quiero que vayan desde el mínimo darkred al máximo darkgreen. Estos son los valores de esa variable

```
Rango_Cat = factor(
  Rango,
  levels = c(1,2,3,4,5),
  labels = c("Deterioro", "Riesgo", "Vulnerable", "Solvente", "Sostenible")
)
```

Para crear este mapa con la variable categórica (el factor de 5 niveles), necesitamos cambiar la escala de una continua (gradiente) a una discreta (manual). Dado que ya tienes definidos los niveles y etiquetas del factor, usaremos scale_fill_manual para asignar los colores exactos de forma secuencial.

Aquí tienes el código ajustado para el nuevo mapa:

```
R
library(ggplot2)
library(plotly)

# 1. Definimos la paleta de 5 colores de rojo oscuro a verde oscuro
colores_rango <- c(
  "Deterioro" = "darkred",
  "Riesgo"    = "firebrick3",
  "Vulnerable" = "gold",
  "Solvente"  = "seagreen3",
  "Sostenible" = "darkgreen"
)

# 2. Creamos el objeto ggplot
p_rango <- ggplot(data = sf_data) +
  geom_sf(aes(fill = Rango_Cat,
```

```

    text = paste("Departamento:", DPTO_CNMBRE,
                 "<br>Categoría:", Rango_Cat,
                 "<br>Valor IDF:", IDF_dep)),
    color = "lightgray",
    size = 0.1) +

# Aplicamos la escala manual para los 5 niveles del factor
scale_fill_manual(values = colores_rango, name = "Categoría de Desempeño") +

labs(title = "Rango de Desempeño Fiscal por Departamento - 2024") +

# Estética solicitada anteriormente
theme_void() +
theme(
  panel.background = element_rect(fill = "white", color = NA),
  plot.title = element_text(hjust = 0.5, face = "bold"),
  legend.position = "right"
)

# 3. Convertimos a interactivo con la nota de fuente abajo a la izquierda
mapa_rango_interactivo <- ggplotly(p_rango, tooltip = "text") %>%
  layout(
    annotations = list(
      list(
        x = 0,
        y = -0.05,
        text = "Fuente: DNP",
        showarrow = FALSE,
        xref = 'paper',
        yref = 'paper',
        xanchor = 'left',
        yanchor = 'top',
        font = list(size = 10, color = "gray")
      )
    ),
    margin = list(b = 50)
  )

# Visualizar
mapa_rango_interactivo

```

¿Qué cambió en esta versión?

1. **scale_fill_manual:** Al ser Rango_Cat una variable categórica (factor), ya no usamos un gradiente, sino que asignamos un color específico a cada una de tus 5 etiquetas. He incluido tonos intermedios para que la transición de "Deterioro" a "Sostenible" sea visualmente fluida.
2. **Paleta de colores:** He definido colores_rango usando nombres de colores de R que van desde el darkred hasta el darkgreen, pasando por un amarillo (gold) para el nivel "Vulnerable".
3. **Tooltip:** He actualizado el texto que aparece al pasar el mouse para que la "Categoría" sea el dato principal a mostrar junto con el nombre del departamento.

Asegúrate de que el objeto sf_data\$Rango_Cat esté efectivamente convertido a factor con el código que compartiste antes de correr el gráfico para que los colores se asignen en el orden correcto.

Cómo puedo pasar este código a leaflet para que sea más óptimo el renderizado? Quiero que igual siga con hover para que me muestre la información respectiva de cada municipio:

```

# 2. Creamos el objeto ggplot
p_rango_mun <- ggplot(data = sf_dataM) +
  geom_sf(aes(fill = Rango_Cat,
text = paste("Municipio:", mpio_cnmb,
"<br>Categoría:", Rango_Cat,
"<br>Valor IDF:", IDF)),
color = "lightgray",
size = 0.1) +

# Aplicamos la escala manual para los 5 niveles del factor
scale_fill_manual(values = colores_rango, name = "Categoría de Desempeño") +

labs(title = "Rango de Desempeño Fiscal por Municipio - 2024") +

# Estética solicitada anteriormente
theme_void() +
theme(
panel.background = element_rect(fill = "white", color = NA),
plot.title = element_text(hjust = 0.5, face = "bold"),
legend.position = "right"
)

# 3. Convertimos a interactivo con la nota de fuente abajo a la izquierda
mapa_rango_mun <- ggplotly(p_rango_mun, tooltip = "text") %>%
  layout(
    # Título centrado
    title = list(
      text = "Índice de Desempeño Fiscal por Municipio - 2024",
      x = 0.5
    ),
    # Añadimos la nota de pie de página como una anotación
    annotations = list(
      list(
        x = 0, # 0 es el extremo izquierdo
        y = -0.05, # Valor negativo para que baje del eje del mapa
        text = "Fuente: DNP",
        showarrow = FALSE,
        xref = 'paper',
        yref = 'paper',
        xanchor = 'left',
        yanchor = 'top',
        font = list(size = 10, color = "gray")
      )
    ),
    # Ajustamos el margen inferior para que la nota no se corte
    margin = list(b = 50)
  )

# Visualizar
mapa_rango_mun

```

Pasar de ggplotly a leaflet es una excelente decisión para optimizar el rendimiento. ggplotly tiende a volverse lento cuando manejas muchos polígonos (como los más de 1,100 municipios de Colombia) porque renderiza cada vértice como un objeto individual de Plotly.

Leaflet, en cambio, utiliza renderizado basado en canvas o SVG optimizado, lo que hace que el **zoom** y el **hover** sean mucho más fluidos.

Aquí tienes el código optimizado:

R

```
library(leaflet)
library(sf)
library(magrittr)
```

```
# 1. Preparación de la paleta de colores
```

```
# Asegúrate de que 'colores_rango' sea un vector con nombre o coincida con los niveles de Rango_Cat
```

```
pal <- colorFactor(palette = colores_rango, domain = sf_dataM$Rango_Cat)
```

```
# 2. Definición del texto del Hover (Labels)
```

```
# Usamos HTML para que los saltos de línea funcionen
```

```
labels <- sprintf(
  "<strong>Municipio:</strong> %s<br/><strong>Categoría:</strong> %s<br/><strong>Valor IDF:</strong> %g",
  sf_dataM$mpio_cnmb, sf_dataM$Rango_Cat, sf_dataM$IDF
) %>% lapply(htmltools::HTML)
```

```
# 3. Creación del Mapa Leaflet
```

```
mapa_leaflet <- leaflet(sf_dataM) %>%
```

```
  addTiles() %>% # Fondo de mapa (puedes usar addProviderTiles(providers$CartoDB.Positron) para algo más limpio)
```

```
  addPolygons(
    fillColor = ~pal(Rango_Cat),
    weight = 0.5,
    opacity = 1,
    color = "white",
    dashArray = "3",
    fillOpacity = 0.7,
    highlightOptions = highlightOptions(
      weight = 2,
      color = "#666",
      dashArray = "",
      fillOpacity = 1,
      bringToFront = TRUE
    ),
    label = labels,
    labelOptions = labelOptions(
      style = list("font-weight" = "normal", padding = "3px 8px"),
      textsize = "13px",
      direction = "auto"
    )
  ) %>%
```

```
  addLegend(
    pal = pal,
    values = ~Rango_Cat,
    opacity = 0.7,
    title = "Desempeño Fiscal",
```

```

    position = "bottomright"
  ) %>%
  addControl(
    html = "<div style='font-size:14px; font-weight:bold;'>Índice de Desempeño Fiscal por Municipio -
2024</div>",
    position = "topright"
  ) %>%
  addControl(
    html = "<div style='font-size:10px; color:gray;'>Fuente: DNP</div>",
    position = "bottomleft"
  )

# Visualizar
mapa_leaflet

```

¿Por qué esto es más rápido?

1. **Manejo de Geometrías:** Leaflet procesa los objetos sf de forma nativa y eficiente. Si el mapa sigue sintiéndose pesado, te recomiendo usar `ms_simplify` del paquete `rmapshaper` para reducir ligeramente la complejidad de las fronteras de los municipios sin perder la forma.
2. **HighlightOptions:** A diferencia de `ggplotly` que "redibuja", Leaflet usa una capa de resaltado (`highlightOptions`) que cambia el estilo del polígono instantáneamente al pasar el mouse.
3. **Labels vs Tooltips:** Al usar `label` en lugar de `popup`, la información aparece inmediatamente al pasar el cursor (`hover`), lo que replica la experiencia de tu código original pero con mayor suavidad.

Tips adicionales de optimización:

- **Simplificación:** Si notas lag al mover el mapa, aplica esto antes de graficar:

```

R
sf_dataM <- rmapshaper::ms_simplify(sf_dataM, keep = 0.05) # Mantiene el 5% de los puntos

```

- **Proyección:** Asegúrate de que `sf_dataM` esté en coordenadas geográficas (WGS84, EPSG:4326), que es el estándar de Leaflet:

```

R
sf_dataM <- st_transform(sf_dataM, 4326)

```